SignBridge

Bridging the Gap in Tunisian Communication

Project Report

A bidirectional AI accessibility platform for the Tunisian Deaf Community

Tunisian Dialect (Darija) $\longleftrightarrow$ *Tunisian Sign Language (TSL)*

Culturally Aware · Real-Time · Hyper-Visual

April 10, 2026

Contents

1 Project Overview

SignBridge is a comprehensive accessibility platform designed to break communication barriers between the deaf and hearing communities in Tunisia. It leverages state-of-the-art deep learning models to provide seamless bidirectional translation between **Tunisian Dialect (Darija)** and **Tunisian Sign Language (TSL)**.

The platform is built on three core principles:

- **Culturally Aware** — Trained on Tunisian speech and sign datasets.
- **Real-Time** — Capable of live sign recognition and instant speech synthesis.
- **Hyper-Visual** — Utilizing neon-skeleton avatars for better clarity and engagement.

System Architecture

The diagram below illustrates the bidirectional communication pipeline. A deaf user's signs are captured, converted to text, and then synthesized into audio for a hearing user. In the reverse direction, the hearing user's speech is transcribed and rendered as an animated avatar for the deaf user.

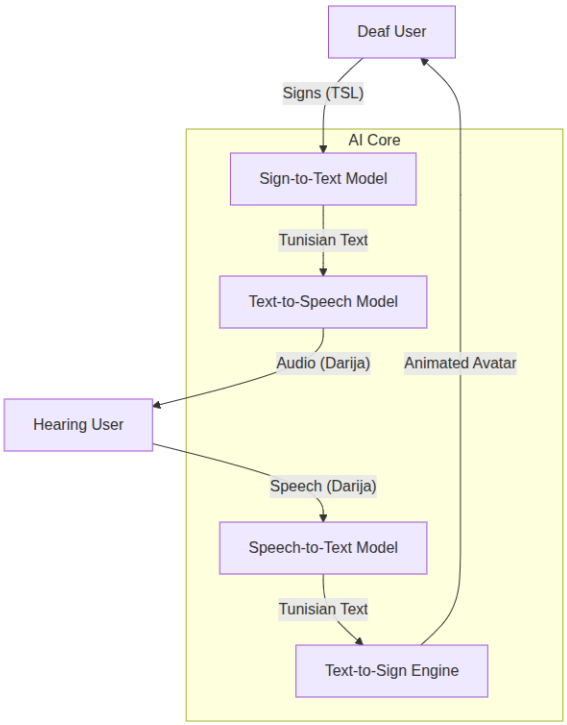

Figure 1: Architecture

Core AI Modules

Module	Technology	Function
Sign to Text (SiTT)	Mediapipe + Keras Temporal Attention	Live hand-gesture classification into TSL signs
Text to Sign (TTSi)	Concatenative Synthesis + Mediapipe Holistic	Renders signs as a Neon Skeleton Avatar
Text to Speech (Tun_TTS)	Coqui XTTS v2 (fine-tuned)	Natural Tunisian Darija voice synthesis
Speech to Text (Tun_STT)	OpenAI Whisper-small + bad-word filter	Darija audio transcription with content safety

2 Module 1 — Sign to Text (SiTT)

2.1 Feature Extraction

Instead of processing raw pixels — which is computationally expensive and sensitive to lighting — the module uses **Mediapipe Hand Landmarker** to extract 21 three-dimensional coordinates (x, y, z) per hand from each video frame.

The landmarks are normalized through two steps. First, all coordinates are shifted relative to the wrist (landmark 0), making the representation translation-invariant. Second, the coordinates are scaled by the maximum distance from the wrist, ensuring invariance to camera distance. In addition, the cosine similarity between vectors formed by the wrist and each fingertip is computed, providing a robust and orientation-independent descriptor

of hand shape.

2.2 Temporal Modeling

Sign language is inherently temporal — a single static frame is often ambiguous. The module addresses this with a windowed approach: sequences of **16 consecutive frames** (approximately 0.5 to 1 second of motion) are maintained in a sliding window buffer and passed together to the classifier.

2.3 Model Architecture

The core classifier uses a **Temporal Attention** mechanism implemented in Keras. Not all frames in a 16-frame window contribute equally — some capture the core hand-shape of a sign while others are transitional. The attention layer learns to assign higher weights to the most discriminative frames, significantly improving accuracy over simple pooling. The model is trained on a custom TSL dataset of over 100 common Tunisian signs across four categories: Demandes, Destinations, Famille, and Transport.

2.4 Platform Integration

Live streaming is handled by `streamlit-webrtc` for zero-latency camera capture. Inference occurs every 8 frames to maintain a high frame rate while delivering smooth predictions. A configurable confidence threshold (default 0.55) filters low-confidence predictions to prevent flickering and false positives.

Important: The training notebook must be run on **Google Colab** with a GPU runtime of at least 10 GB of VRAM. TensorFlow/Keras and Mediapipe come pre-installed in the Colab environment.

3 Module 2 — Text to Sign Avatar (TTSi)

3.1 Data-Driven Concatenative Synthesis

Unlike standard 3D avatars that require complex rigging, the TTSi module is fully data-driven. A dataset of native Tunisian signers was recorded and processed frame-by-frame using **Mediapipe Holistic**, extracting 468 face landmarks, 33 pose landmarks, and 21 landmarks per hand simultaneously. When a user inputs text, the engine tokenizes it into individual words and retrieves the corresponding pre-extracted landmark sequences, which are then chained into a continuous animation.

3.2 The Neon Skeleton Rendering Engine

The visual output is produced by a custom **HTML5 Canvas renderer**. Three-dimensional normalized coordinates from Mediapipe are mapped to a 400×560 pixel canvas. Each body segment is rendered in a distinct color to maximize clarity, particularly during overlapping hand movements. Z-coordinate values are used to convey depth through glow intensity.

Body Part	Visual Style	Hex Code
Pose (Body)	Indigo with subtle glow	`#7C6AF7`
Right Hand	Pink with intensive neon glow	`#F7A6C1`
Left Hand	Teal for overlap clarity	`#6AF7D4`
Head	Gradient radial circle	Pose-tracked

Table 1: Visual encoding of body segments in the Neon Skeleton renderer.

3.3 Design Philosophy: Clarity over Realism

The Neon Skeleton approach deliberately avoids photorealistic 3D avatars, which can suffer from the "uncanny valley" effect and distract from the actual hand shapes. It eliminates visual noise (clothes, facial details, background), emphasizes hand trajectory and movement, and is web-optimized — landmark data stored as JSON is far lighter than raw video, enabling fast loading on mobile devices.

3.4 Playback System

A speed slider allows users to control animation from 1 to 15 FPS, making the module a practical learning tool for those studying TSL. The engine also supports multi-word chaining, combining the landmark sequences of several words into one seamless animation.

Note: TTSi requires no training notebook. It uses pre-extracted landmark data directly — simply launch the demo and navigate to the Avatar Synthesis page.

4 Module 3 — Text to Speech (Tun_TTS)

4.1 Base Model: Coqui XTTS v2

The module is built on **Coqui XTTS v2**, an auto-regressive transformer-based TTS model using a GPT-like architecture. The model predicts audio tokens from text input, which are then decoded into high-fidelity waveforms using a **HiFi-GAN** vocoder. Its multilingual pre-training on Modern Standard Arabic (MSA) provides a strong foundation for fine-tuning on Tunisian Darija. It also supports zero-shot voice cloning from a 6-second reference audio clip.

4.2 Fine-Tuning on TunArTTS

The model was fine-tuned on the **TunArTTS Corpus**: approximately 3 hours of high-quality Tunisian speech from multiple speakers. The corpus uses diacritized Arabic script (tashkeel), which is vital for disambiguating Darija words that share the same root but differ in vowels. Training ran for 2,000+ steps on a Tesla T4 GPU, optimizing the GPT encoder weights while keeping the HiFi-GAN vocoder frozen to preserve audio clarity.

4.3 Tunisian Text Normalization

A significant challenge in Tunisian TTS is the lack of spelling standardization — the same word may be written in many ways. The custom pre-processing pipeline handles three types of normalization: spelling standardization (mapping orthographic variants to a canonical form), phonetic mapping of French loanwords to their Arabic phonetic equivalents (ensuring natural Tunisian pronunciation), and number expansion (converting digits to their Tunisian spoken form, e.g., “2” becomes the Tunisian word for two).

4.4 Performance & Quality

The fine-tuned model outperforms vanilla MSA models in three areas: prosody (capturing the distinct rhythm of Tunisian speech), vocabulary (correctly pronouncing dialect-specific particles such as *baahi* and *barsha*), and voice cloning (zero-shot speaker synthesis from a short reference clip).

Important: The fine-tuning notebook must be run on **Google Colab** with at least 10 GB of VRAM. Coqui TTS and PyTorch come pre-installed in the Colab environment.

5 Module 4 — Speech to Text (Tun_STT)

5.1 The Engine: OpenAI Whisper-small

The module uses **Whisper-small**, OpenAI's open-source ASR model trained on 680,000 hours of multilingual audio. Its encoder-decoder Transformer architecture jointly performs speech recognition and language identification, making it naturally suited to the Arabic-French code-switching characteristic of Tunisian speech. Its large-scale multilingual pretraining allows it to generalize well to dialect-specific phonemes, including the Qaf/Gaf distinction and the specific vowel sounds of Darija. Training on diverse audio conditions also provides robustness to background noise typical of mobile environments such as street noise and café chatter.

5.2 Processing Pipeline

All input audio is internally resampled to 16 kHz mono. The signal is then converted into an 80-channel log-Mel spectrogram and passed to the encoder. The decoder produces the final transcript using beam search for high accuracy.

5.3 Tunisian Bad-Word Filter

After transcription, the output passes through a profanity filter built specifically for Tunisian Darija, using a curated `.xlsx` dataset of known Tunisian profane words and expressions. Each transcribed token is checked against the dataset, and matches are flagged or censored before display. Standard Arabic profanity lists do not cover Tunisian dialect slang, making this custom filter essential for culturally accurate content moderation.

Property	**Value**
Model size	≈ 461 MB (244M parameters)
Training data	680,000 hours of multilingual audio
Internal sample rate	16 kHz mono
Content safety	Tunisian bad-word filter (.xlsx dataset)
Environment	Linux, Windows, macOS — CUDA supported

Table 2: Whisper-small model characteristics.

Important: The notebook must be run on **Google Colab** with at least 10 GB of VRAM. Whisper-small is downloaded automatically on first run via `whisper.load_model("small")`.

6 Streamlit Demo Suite

All four modules are integrated into a multi-page **Streamlit** dashboard located in the `/Demo` directory.

#	Page	Description
1	TTS Studio	Advanced text-to-speech generation with sample Tunisian phrases.
2	STT Workspace	Upload or record audio for instant transcription using Whisper-small.
3	Sign Recognition	Live webcam feed for real-time Tunisian sign detection.
4	Avatar Synthesis	Enter text (e.g., "3aslema") to see the neon avatar perform the sign.
5	Conversation Bridge	A unified interface for bidirectional deaf ↔ hearing interaction.

Table 3: Pages in the Streamlit demo application.

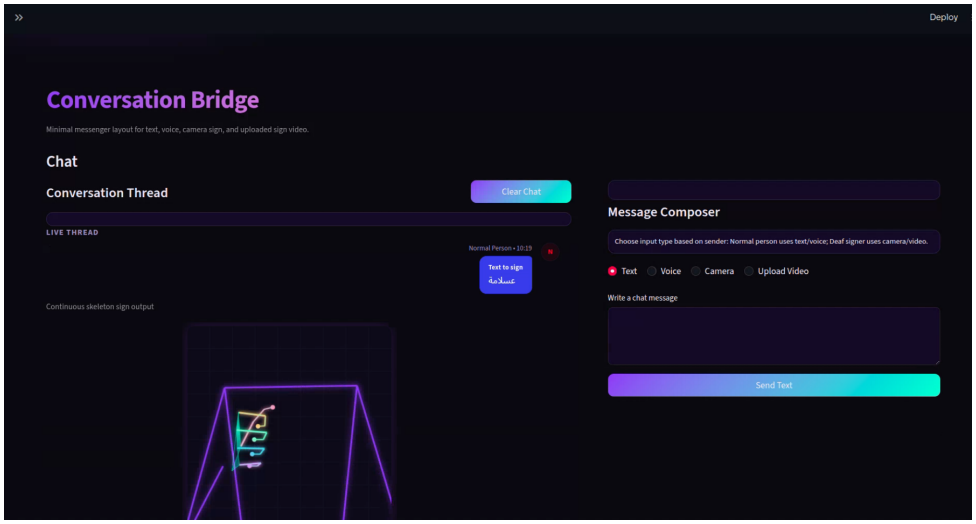

Figure 2: Chat text

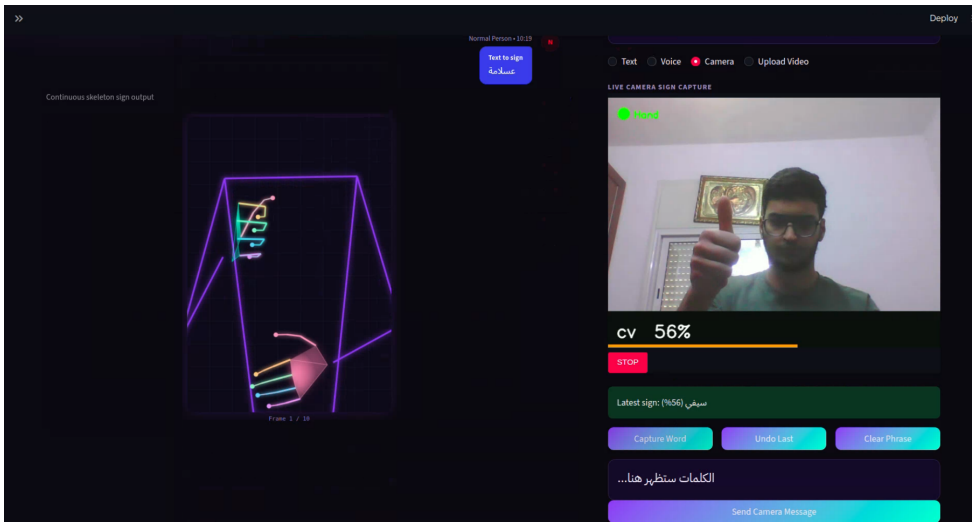

Figure 3: Sign to speech

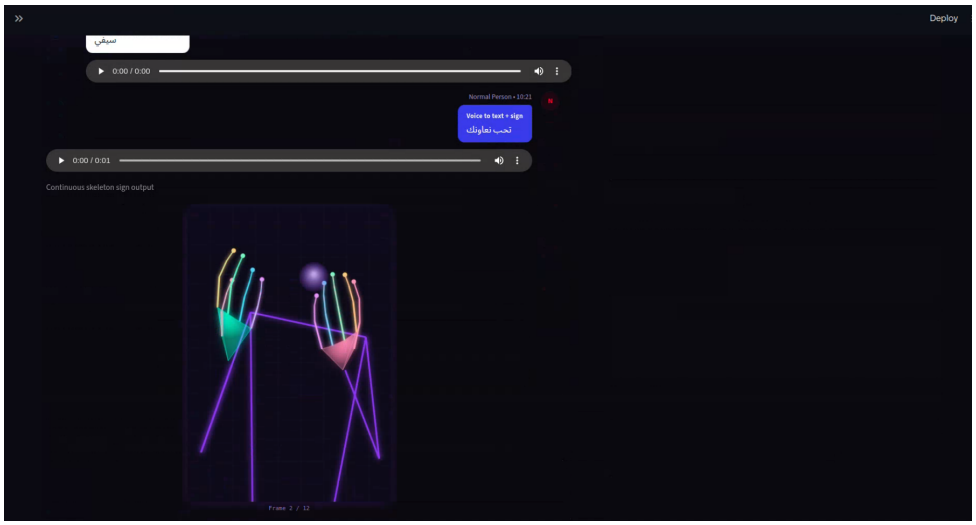

Figure 4: speech to sign